Universidad Nacional Autónoma de México

Facultad de Ciencias

Compiladores

Semestre 2026–2

Fecha de entrega 07/04/2026

Práctica 2

Alumno:

- Carrillo Gonzalez Luis Israel – 321050324

1. Ejercicio 1

Los símbolos no terminales son:

$N = \{programa, declaraciones, declaracion, tipo, lista_var, sentencias, sentencia, expresion\}$

Los símbolos terminales son:

$\Sigma = \{int, float, identificador, numero, if, else, while, ;, ,, =, (,), +, -, *, /\}$

Símbolo inicial:

$$S = programa$$

2. Ejercicio 2

En la gramática dada sí hay ambigüedad; esta es ocasionada por el símbolo no terminal "expresión" debido a sus producciones.

Para eliminarla tenemos que eestructurar las producciones correspondientes a "expresion" introduciendo nuevos símbolos no terminales.

Las nuevas producciones serían:

- $+expresion \rightarrow +expresion + *expresion| + expresion - *expresion| * expresion$
- $*expresion \rightarrow *expresion*()expresion|*expresion/()expresion|()expresion$
- $()expresion \rightarrow identificador|numero|(+expresion)$

3. Ejercicio 3

La gramática si tiene recursividad izquierda, la gramática sin esta es:

- $programa \rightarrow declaraciones \quad sentencias$
- $declaraciones \rightarrow declaracion \quad declaraciones'$
- $declaraciones' \rightarrow declaracion \quad declaraciones'|\epsilon$
- $declaracion \rightarrow tipo \quad lista - var \quad ;$
- $tipo \rightarrow int|float$
- $lista - var \rightarrow identificador \quad lista - var'$

- $lista - var' \rightarrow, \quad identificador \quad lista - var' | \epsilon$
- $sentencias \rightarrow sentencia \quad sentencias'$
- $sentencias' \rightarrow sentencia \quad sentencias' | \epsilon$
- $sentencia \rightarrow identificador = expresion; |$
  $if(expresion) \quad sentencias \quad else \quad sentencias |$
  $while \quad (expresion) \quad sentencias$
- $+expresion \rightarrow *expresion \quad + expresion'$
- $+expresion' \rightarrow + \quad *expresion \quad +expresion | - \quad *expresion \quad +expresion' | \epsilon$
- $*expresion \rightarrow ()expresion \quad * expresion'$
- $*expresion' \rightarrow * \quad ()expresion \quad *expresion' | / \quad ()expresion \quad *expresion' | \epsilon$
- $()expresion \rightarrow identificador | numero | (+expresion)$

4. Ejercicio 4

Como en ninguna parte de la gramática existe un símbolo no terminal que tenga dos o más producciones que empiecen con el mismo símbolo, entonces la grámatica no se puede factorizar.

5. Ejercicio 5

$N = \{programa, declaraciones, declaraciones', declaracion, tipo, lista_var, lista_var', sentencias, sentencias', sentencia, +expresion, +expresion', *expresion, *expresion', ()expresion\}$

El conjunto P es el siguiente:

- $programa \rightarrow declaraciones\ sentencias$
- $declaraciones \rightarrow declaracion\ declaraciones'$
- $declaraciones' \rightarrow declaracion\ declaraciones' | \epsilon$
- $declaracion \rightarrow tipo\ lista_var\ ;$
- $tipo \rightarrow int | float$
- $lista_var \rightarrow identificador\ lista_var'$
- $lista_var' \rightarrow,\ identificador\ lista_var' | \epsilon$
- $sentencias \rightarrow sentencia\ sentencias'$

- $sentencias' \rightarrow sentencia\ sentencias' | \epsilon$
- $sentencia \rightarrow identificador\ =\ +expresion\ ;$
- $sentencia \rightarrow identificador\ =\ +expresion\ ; |$
  $if\ (\ +expresion\ )\ sentencias\ else\ sentencias |$
  $while\ (\ +expresion\ )\ sentencias$
- $+expresion \rightarrow *expresion\ +expresion'$
- $+expresion' \rightarrow +\ *expresion\ +expresion' | -\ *expresion\ +expresion' | \epsilon$
- $*expresion \rightarrow ()expresion\ *expresion'$
- $*expresion' \rightarrow *\ ()expresion\ *expresion' | /\ ()expresion\ *expresion' | \epsilon$
- $()expresion \rightarrow identificador | numero | (\ +expresion\ )$